

“AWS Cloud Zero to hero”



Table of Content –

1. Creating a AWS Free Tier account
2. Learning AWS Free on AWS Skill builder website with “AWS Cloud Practitioner”
3. Step by Step guide to University Live project
4. Resume Write up based on above project
5. Brief recap of Important AWS Services to clear the Interview

Roadmap to learn AWS cloud is just 30 days -

First step – Create your AWS Free Tier account as below:

Step 1 - Reinvent how you work with IT by signing up for the [AWS Free Tier](#), which allows you to gain hands-on experience with a broad selection of AWS products and services.

By [signing up for AWS](#), you have access to Amazon cloud computing services.

Please Note

The sign-up process requires a credit card, which will not be charged until you start using services. There are no long-term commitments and you can stop using AWS at any time.

Step 2 - Learn AWS for Free from the AWS Cloud Practitioner essential course below -

<https://explore.skillbuilder.aws/learn/course/134/aws-cloud-practitioner-essentials>

Step 3 - Get hands-on experience from our [self-paced labs](#).



AWS Learning Roadmap – 30 Days

Table of Contents:

1. Day 1-3: Introduction to Cloud Computing & AWS Basics
2. Day 4-6: AWS Identity and Access Management (IAM)
3. Day 7-9: Amazon S3 (Simple Storage Service)
4. Day 10-12: Amazon EC2 (Elastic Compute Cloud)
5. Day 13-15: Amazon RDS (Relational Database Service)
6. Day 16-17: AWS Lambda
7. Day 18-19: Amazon CloudFront
8. Day 20-21: Amazon VPC (Virtual Private Cloud)
9. Day 22-23: AWS Elastic Load Balancer (ELB)
10. Day 24-25: Amazon DynamoDB
11. Day 26-27: Amazon SNS (Simple Notification Service)
12. Day 28-30: AWS CloudWatch

Day 1-3: Introduction to Cloud Computing & AWS Basics

- **Objective:** Understand the basics of cloud computing, cloud models (IaaS, PaaS, SaaS), and AWS's global infrastructure.
- **Learning Resources:**
 - [AWS Cloud Basics](#)
 - [AWS Global Infrastructure](#)
- **Diagram:** Visualize cloud models and the architecture of AWS's global infrastructure.

Day 4-6: AWS Identity and Access Management (IAM)

- **Service:** IAM manages access to AWS services and resources securely. You can create and manage users, groups, and roles.
- **Diagram:** IAM architecture showing users, roles, policies, and groups.
- **More Info:** [AWS IAM](#)

Day 7-9: Amazon S3 (Simple Storage Service)

- **Service:** Amazon S3 offers scalable object storage for data, including backups, website assets, and big data analytics.
- **Diagram:** S3 bucket containing objects such as images, documents, and videos.
- **More Info:** [Amazon S3](#)

Day 10-12: Amazon EC2 (Elastic Compute Cloud)

- **Service:** EC2 allows resizable compute capacity in the cloud. You can scale instances up or down based on demand.
- **Diagram:** EC2 instance showing scaling, auto-scaling groups, and instance types.
- **More Info:** [Amazon EC2](#)

Day 13-15: Amazon RDS (Relational Database Service)

- **Service:** RDS is a managed service that makes database setup, operation, and scaling simple. Supports various database engines like MySQL and PostgreSQL.
- **Diagram:** RDS with automated backups, multi-AZ replication.
- **More Info:** [Amazon RDS](#)

Day 16-17: AWS Lambda

- **Service:** AWS Lambda allows you to run code without provisioning or managing servers. It's event-driven, automatically scaling up or down.
- **Diagram:** Lambda function triggered by events like S3 file upload.
- **More Info:** [AWS Lambda](#)

Day 18-19: Amazon CloudFront

- **Service:** CloudFront delivers content globally with low latency through its CDN. It caches data at edge locations close to the user.
- **Diagram:** CloudFront with edge locations and an origin server.
- **More Info:** [Amazon CloudFront](#)

Day 20-21: Amazon VPC (Virtual Private Cloud)

- **Service:** VPC allows you to launch AWS resources in a virtual network you define. Control over IP addressing, subnets, and gateways.
- **Diagram:** VPC with public and private subnets, internet gateway, NAT gateway.
- **More Info:** [Amazon VPC](#)

Day 22-23: AWS Elastic Load Balancer (ELB)

- **Service:** ELB distributes incoming traffic across multiple instances, improving availability and fault tolerance.
- **Diagram:** ELB distributing traffic across EC2 instances.
- **More Info:** [AWS ELB](#)

Day 24-25: Amazon DynamoDB

- **Service:** DynamoDB is a managed NoSQL database that provides fast and predictable performance with seamless scalability.
- **Diagram:** DynamoDB table showing items, attributes, partition keys.
- **More Info:** [Amazon DynamoDB](#)

Day 26-27: Amazon SNS (Simple Notification Service)

- **Service:** SNS allows you to send messages to a large number of subscribers via email, SMS, or mobile push notifications.
- **Diagram:** SNS topic distributing messages to email, SMS subscribers.
- **More Info:** [Amazon SNS](#)

Day 28-30: AWS CloudWatch

- **Service:** CloudWatch monitors AWS resources and applications in real time. You can set alarms and

- create dashboards for metric visualization.
- **Diagram:** CloudWatch with custom metrics, alarms, and log groups.
- **More Info:** [AWS CloudWatch](#)

University level Live Project to add into your resume -

Here's a project write-up that combines **SQL**, **Python**, and the **top 10 AWS services** you are focusing on, which can be presented as a university project:

Title: University Course Data Analysis as an AWS Data Engineer

As part of my university project, I was tasked with building an end-to-end data pipeline to analyze course enrollment trends and student performance using AWS services. The project focused on automating data ingestion, processing, and generating insights to assist university administrators in optimizing course offerings and academic resources.

Project Overview:

The goal of the project was to design a scalable data pipeline that could handle large volumes of university course data, including:

- Student enrollment records for each course.
- Performance data (grades) across different semesters.
- Feedback from students regarding course content and instructors.

Steps Involved:

1. **Data Ingestion Using Amazon S3:** I started by uploading the university course data, stored as CSV files, to **Amazon S3**. S3 served as the centralized storage for raw and processed data. I also automated data ingestion from the university's database, scheduling regular uploads of new data using AWS Lambda functions and S3 event triggers.
2. **Data Processing with AWS Lambda and AWS Glue:** To clean and transform the raw data, I used **AWS Lambda** to trigger Python-based ETL processes. The Lambda functions were designed to extract data from S3, clean and normalize it, and then load the processed data into an **Amazon RDS** MySQL database for further analysis.

I also explored **AWS Glue** for serverless ETL processing. Glue enabled me to create and run data transformation jobs efficiently, allowing me to handle larger datasets and ensure that the data was properly structured for analysis.

3. **Database Setup with Amazon RDS:** I used **Amazon RDS** to set up a relational database that stored the cleaned and structured course data. The RDS instance allowed me to perform SQL queries and analysis on the data, and it was connected to other AWS services for a seamless data flow.

4. **Data Analysis and Visualization:** Once the data was processed and stored in RDS, I used **Python** (via Jupyter notebooks) to run SQL queries and analyze the university course data. I focused on:
- Identifying trends in course enrollment over semesters.
 - Analyzing grade distributions across departments.
 - Evaluating student feedback on instructors.

I used **Amazon QuickSight** to create dashboards for visualizing insights, which were shared with university stakeholders. The visualizations helped identify courses that needed additional support and optimization in terms of scheduling and instructor assignment.

5. **Automation and Monitoring with AWS CloudWatch:** To ensure smooth pipeline operation, I set up **AWS CloudWatch** for monitoring and alerts. This helped track the performance of Lambda functions, S3 storage usage, and RDS query performance, allowing me to maintain the pipeline and troubleshoot issues quickly.

Results and Insights:

The project provided valuable insights into course enrollment patterns, including:

- Identifying high-demand courses that required additional sections.
- Highlighting courses where student performance was consistently low, indicating a need for improved academic support.
- Revealing trends in student feedback that informed adjustments to course content and teaching methods.

This AWS-based data engineering project helped me build scalable solutions to process large datasets and gain meaningful insights, aligning my skills with industry standards for cloud-based data engineering.

Here's a step-by-step guide on how the university course data analysis project was built, using Python, SQL, and various AWS services. I will break it down into the key phases: data ingestion, processing, storage, analysis, and visualization. Each phase will include the relevant AWS services, Python code, and SQL commands.

Step 1: Data Ingestion Using Amazon S3

AWS Service: Amazon S3

Python Module: `boto3`

The first step is to upload raw course data (CSV files) to Amazon S3. This allows you to store large amounts of data and later process it for analysis.

1.1. Create an S3 Bucket:

Log into AWS Management Console and create an S3 bucket to store the university course data.

1.2. Upload Files to S3 Using Python:

Here's a sample Python code using `boto3` to upload CSV files to an S3 bucket.

python

 Copy code

```
import boto3

# Initialize S3 client
s3 = boto3.client('s3')

# File and bucket information
bucket_name = 'university-course-data'
file_name = 'course_data.csv'
file_path = '/path/to/local/course_data.csv'

# Upload the file to S3
s3.upload_file(file_path, bucket_name, file_name)
print(f'{file_name} has been uploaded to {bucket_name}')
```

Step 2: Data Processing Using AWS Lambda and AWS Glue

AWS Service: AWS Lambda, AWS Glue

Python Module: pandas for data cleaning and transformation

You can either use Lambda for small-scale transformations or AWS Glue for more complex ETL (Extract, Transform, Load) jobs.

2.1. Using AWS Lambda for ETL:

1. Create a Lambda function from the AWS Console.
2. Write Python code to extract, clean, and upload processed data back to S3 or a database.

Here's a simplified Lambda function that reads a CSV file from S3, processes it using Pandas, and uploads the cleaned data back to S3:

```
python
import json
import boto3
import pandas as pd

def lambda_handler(event, context):
    s3 = boto3.client('s3')

    # Define S3 bucket and file details
    bucket = 'university-course-data'
    file_key = 'raw/course_data.csv'

    # Download CSV file from S3
    response = s3.get_object(Bucket=bucket, Key=file_key)
    data = response['Body'].read().decode('utf-8')

    # Load data into Pandas dataframe
    df = pd.read_csv(data)

    # Data cleaning
    df['grade'] = df['grade'].fillna('N/A')
    df_cleaned = df.dropna(subset=['course_id'])

    # Save cleaned data back to S3
    cleaned_file_key = 'processed/course_data_cleaned.csv'
    df_cleaned.to_csv('/tmp/course_data_cleaned.csv', index=False)
    s3.upload_file('/tmp/course_data_cleaned.csv', bucket, cleaned_file_key)

    return {
        'statusCode': 200,
        'body': json.dumps('Data processed and uploaded to S3')
    }
```

Step 3: Storing Data Using Amazon RDS

AWS Service: Amazon RDS (MySQL/PostgreSQL)

For structured data, we'll use an Amazon RDS instance (MySQL) to store the cleaned course data. This allows you to run SQL queries for further analysis.

3.1. Set Up Amazon RDS:

1. Create an RDS instance in the AWS Console.
2. Set up your database with MySQL or PostgreSQL.
3. Allow access to the database from your Lambda function or local machine.

3.2. Create a Table in MySQL:

Once RDS is set up, connect to the MySQL instance and create a table to store the cleaned data:


```
sql

CREATE TABLE course_data (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(255),
  department VARCHAR(255),
  instructor VARCHAR(255),
  student_count INT,
  avg_grade FLOAT,
  feedback_score FLOAT
);
```

Step 4: Load Data into RDS Using Python

Use Python with the pymysql library to connect to the RDS instance and insert the cleaned data.

```
python
```

```
Copy code
```

```
import pymysql
```

```
import pandas as pd
```

```
# RDS connection info
```

```
host = 'your-rds-endpoint'
```

```
user = 'admin'
```

```
password = 'password'
```

```
db = 'university_data'
```

```
# Connect to RDS MySQL
```

```
conn = pymysql.connect(host=host, user=user, password=password, db=db)
```

```
# Load cleaned data from CSV
```

```

df_cleaned = pd.read_csv('/path/to/cleaned_data.csv')

# Insert data into the RDS table
cursor = conn.cursor()

for index, row in df_cleaned.iterrows():
    cursor.execute(
        'INSERT INTO course_data (course_id, course_name, department, instructor, student_count, avg_grade,
        feedback_score) VALUES (%s, %s, %s, %s, %s, %s, %s)',
        (row['course_id'], row['course_name'], row['department'], row['instructor'], row['student_count'], row['avg_grade'],
        row['feedback_score'])
    )

conn.commit()
cursor.close()
conn.close()

```

Step 5: Data Analysis Using SQL and Python

Now that the data is in RDS, you can run SQL queries to analyze it. Here's an example of using SQL to analyze course enrollment and student performance.

```

sql

-- Query to find courses with the highest enrollment
SELECT course_name, student_count
FROM course_data
ORDER BY student_count DESC
LIMIT 10;

-- Query to find average grades by department
SELECT department, AVG(avg_grade) AS avg_grade
FROM course_data
GROUP BY department;

```

Additionally, you can analyze the data in Python using pandas and visualize it using matplotlib or

Step 6: Visualizing Data Using Amazon QuickSight

AWS Service: **Amazon QuickSight**

Set up Amazon QuickSight **and connect it to your RDS database.**

Create dashboards to visualize course enrollment trends, grade distributions, and instructor performance.

Some useful visualizations include:

Bar charts **for enrollment across departments.**

Line charts **to show grade trends over semesters.**

Heatmaps **to correlate instructor feedback with student performance.**

Conclusion:

This project illustrates the use of Python, SQL, and AWS services (S3, Lambda, Glue, RDS, and QuickSight) to build an end-to-end data analysis pipeline. By leveraging AWS services, you can process, store, and analyze large datasets efficiently, making the project scalable and adaptable to real-world applications.

This setup can easily be expanded to other datasets or integrated with machine learning models using services like Amazon SageMaker to predict future course enrollment trends or student performance.

KAVITHA LAKSHMINARASAIAH

Resume Write up –

University Course Data Analysis Project

Summary: Data Engineer with experience building scalable data pipelines using AWS services to process, analyze, and visualize large datasets. Successfully executed a project to analyze course enrollment and student performance data for a university, delivering actionable insights to optimize course offerings and academic support. Proficient in AWS Lambda, S3, RDS, Python, SQL, and data visualization using Amazon QuickSight.

Project: University Course Data Analysis

Objective: Built an end-to-end data pipeline to automate the ingestion, processing, storage, and analysis of university course data, including enrollment records, grades, and instructor feedback.

Key Accomplishments:

- Designed a data pipeline that processed 500,000+ rows of raw course data using AWS Lambda and AWS Glue, reducing manual ETL time by 70%.
- Automated data ingestion using Amazon S3, handling daily uploads of 10 GB of course-related data from the university's systems.
- Optimized data cleaning processes in Python (Pandas), improving the accuracy of grade and enrollment records by 95%.
- Integrated Amazon RDS (MySQL) to store cleaned data, enabling the execution of complex SQL queries that provided insights into student performance and course demand.
- Developed 5+ SQL queries that identified high-enrollment courses with low performance, leading to a 30% increase in student support services for those courses.
- Created 5 dashboards using Amazon QuickSight to visualize enrollment trends, grade distributions, and instructor performance, which were presented to university stakeholders.
- Achieved a 20% improvement in course allocation decisions based on data insights, optimizing academic resource distribution across departments.

Technical Skills:

- **AWS Services:** Amazon S3, AWS Lambda, AWS Glue, Amazon RDS, Amazon QuickSight, AWS CloudWatch
- **Programming Languages:** Python (Pandas, Matplotlib), SQL (MySQL, PostgreSQL)
- **Data Processing Tools:** Python (Pandas, NumPy), SQL
- **Data Visualization:** Amazon QuickSight, Matplotlib, Seaborn
- **ETL & Automation:** Serverless architecture using AWS Lambda, automated data workflows

What is cloud computing?

Cloud computing is the on-demand delivery of compute power, database, storage, applications, and other IT resources through a cloud services platform via the internet with pay-as-you-go pricing. Whether you are running applications that share photos to millions of mobile users or you're supporting the critical operations of your business, a cloud services platform provides rapid access to flexible and low-cost IT resources. With cloud computing, you don't need to make large upfront investments in hardware and spend a lot of time on the heavy lifting of managing that hardware. Instead, you can provision exactly the right type and size of computing resources you need to power your newest bright idea or operate your IT department. You can access as many resources as you need, almost instantly, and only pay for what you use.

Cloud computing provides a simple way to access servers, storage, databases and a broad set of application services over the internet. A cloud services platform such as Amazon Web Services owns and maintains the network-connected hardware required for these application services, while you provision and use what you need via a web application.

Six advantages of cloud computing

- **Trade fixed expense for variable expense** – Instead of having to invest heavily in data centers and servers before you know how you're going to use them, you can pay only when you consume computing resources, and pay only for how much you consume.
- **Benefit from massive economies of scale** – By using cloud computing, you can achieve a lower variable cost than you can get on your own. Because usage from hundreds of thousands of customers is aggregated in the cloud, providers such as AWS can achieve higher economies of scale, which translates into lower pay-as-you-go prices.
- **Stop guessing capacity** – Eliminate guessing on your infrastructure capacity needs. When you make a capacity decision prior to deploying an application, you often end up either sitting on expensive idle resources or dealing with limited capacity. With cloud computing, these problems go away. You can access as much or as little capacity as you need, and scale up and down as required with only a few minutes' notice.
- **Increase speed and agility** – In a cloud computing environment, new IT resources are only a click away, which means that you reduce the time to make those resources available to your developers from weeks to just minutes. This results in a dramatic increase in agility for the organization, since the cost and time it takes to experiment and develop is significantly lower.
- **Stop spending money running and maintaining data centers** – Focus on projects that differentiate your business, not the infrastructure. Cloud computing lets you focus on your own customers, rather than on the heavy lifting of racking, stacking, and powering servers.
- **Go global in minutes** – Easily deploy your application in multiple regions around the world with just a few clicks. This means you can provide lower latency and a better experience for your customers at minimal cost.

Types of cloud computing

Cloud computing provides developers and IT departments with the ability to focus on what matters most and avoid undifferentiated work such as procurement, maintenance, and capacity planning. As cloud computing has grown in popularity, several different models and deployment strategies have emerged to help meet specific needs of different users. Each type provides you with different levels of control, flexibility, and management.

Deployment models

Cloud

A cloud-based application is fully deployed in the cloud and all parts of the application run in the cloud. Applications in the cloud have either been created in the cloud or have been migrated from an existing infrastructure to take advantage of the [benefits of cloud computing](#). Cloud-based applications can be built on low-level infrastructure pieces or can use higher level services that provide abstraction from the management, architecting, and scaling requirements of core infrastructure.

Private cloud (on-premises)

The deployment of resources on-premises, using virtualization and resource management tools, is sometimes called the *private cloud*. On-premises deployment doesn't provide many of the benefits of cloud computing but it is sometimes sought for its ability to provide dedicated resources. In most cases, this deployment model is the same as legacy IT infrastructure while using application management and virtualization technologies to try and increase resource utilization. For more information on how AWS can help, refer to [Use case: Cloud services on-premises](#).

Hybrid

A hybrid deployment is a way to connect infrastructure and applications between cloud-based resources and existing resources that are not located in the cloud. The most common method of hybrid deployment is between the cloud and existing on-premises infrastructure to extend, and grow, an organization's infrastructure into the cloud while connecting cloud resources to the internal system. For more information on how AWS can help you with your hybrid deployment, visit our [Hybrid Cloud with AWS](#)

Benefits of AWS security

- **Keep Your data safe** — The AWS infrastructure puts strong safeguards in place to help protect your privacy. All data is stored in highly secure AWS data centers.
- **Meet compliance requirements** — AWS manages dozens of compliance programs in its infrastructure. This means that segments of your compliance have already been completed.

- **Save money** — Cut costs by using AWS data centers. Maintain the highest standard of security without having to manage your own facility
- **Scale quickly** — Security scales with your AWS Cloud usage. No matter the size of your business, the AWS infrastructure is designed to keep your data safe.

KAVITHA LAKSHMINARASIAH

AWS services by category

AWS consists of many cloud services that you can use in combinations tailored to your business or organizational needs. This section introduces the major AWS services by category. Choose a category to explore its services.

Important AWS services that you should definitely know!

AWS Management Console

Access and manage Amazon Web Services through the [AWS Management Console](#), a simple and intuitive user interface. You can also use the [AWS Management Console Application](#) to quickly view resources on the go.

Amazon Simple Storage Service

[Amazon Simple Storage Service](#) (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance. This means customers of all sizes and industries can use it to store and protect any amount of data for a range of use cases, such as websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics. Amazon S3 provides easy-to-use management features so you can organize your data and configure finely-tuned access controls to meet your specific business, organizational, and compliance requirements. Amazon S3 is designed for 99.999999999% (11 9s) of durability, and stores data for millions of applications for companies all around the world.

[Amazon S3 storage classes](#) are a range of storage classes that you can choose from based on the data access, resiliency, and cost requirements of your workloads. S3 storage classes are purpose-built to provide the lowest cost storage for different access patterns. S3 storage classes are ideal for virtually any use case, including those with demanding performance needs, data residency requirements, unknown or changing access patterns, or archival storage.

The S3 storage classes include:

- **S3 Intelligent-Tiering** for automatic cost savings for data with unknown or changing access patterns
- **S3 Standard** for frequently accessed data
- **S3 Express One Zone** for your most frequently accessed data
- **S3 Standard-Infrequent Access (S3 Standard-IA)** and **S3 One Zone-Infrequent Access (S3 One Zone-IA)** for less frequently accessed data
- **S3 Glacier Instant Retrieval** for archive data that needs immediate access

- **S3 Glacier Flexible Retrieval** (formerly S3 Glacier) for rarely accessed long-term data that does not require immediate access
- **Amazon S3 Glacier Deep Archive (S3 Glacier Deep Archive)** for long-term archive and digital preservation with retrieval in hours at the lowest cost storage in the cloud

If you have data residency requirements that can't be met by an existing AWS Region, you can use the **S3 Outposts** storage class to store your S3 data on premises. Amazon S3 also offers capabilities to manage your data throughout its lifecycle. Once an S3 Lifecycle policy is set, your data will automatically transfer to a different storage class without any changes to your application. For more information, refer to the [Amazon S3 storage classes overview info graphic](#).

You can use [S3 Object Lock](#) to help prevent S3 objects from being deleted or overwritten for a fixed amount of time, or indefinitely. Object Lock can help you to meet regulatory requirements that require WORM (write-once-read-many) storage, or to simply add another layer of protection against object changes or deletion.

Amazon EC2

[Amazon Elastic Compute Cloud](#) (Amazon EC2) is a web service that provides secure, resizable compute capacity in the cloud. It is designed to make web-scale computing easier for developers.

The simple web interface of Amazon EC2 allows you to obtain and configure capacity with minimal friction. It provides you with complete control of your computing resources and lets you run on Amazon's proven computing environment. Amazon EC2 reduces the time required to obtain and boot new server instances (called Amazon EC2 instances) to minutes, allowing you to quickly scale capacity, both up and down, as your computing requirements change. Amazon EC2 changes the economics of computing by allowing you to pay only for capacity that you actually use. Amazon EC2 provides developers and system administrators the tools to build failure resilient applications and isolate themselves from common failure scenarios.

Amazon EC2 passes on to you the financial benefits of Amazon scale. You pay a very low rate for the compute capacity you actually consume. For a more detailed description, refer to [Amazon EC2 pricing](#).

[Amazon EC2 instance types](#) are named based on their family, generation, processor family, additional capabilities, and size.

- **On-Demand Instances** — With On-Demand Instances, you pay for compute capacity by the hour or the second depending on which instances you run. No longer-term commitments or upfront payments are needed. You can increase or decrease your compute capacity depending on the demands of your application and only pay the specified per hourly rates for the instance you use. On-Demand Instances are recommended for:
 - Users that prefer the low cost and flexibility of Amazon EC2 without any up-front payment or long-term commitment
 - Applications with short-term, spiky, or unpredictable workloads that cannot be interrupted
 - Applications being developed or tested on Amazon EC2 for the first time

- **Spot Instances** — [Spot Instances](#) are available at up to a 90% discount compared to On-Demand prices and let you take advantage of unused Amazon EC2 capacity in the AWS Cloud. You can significantly reduce the cost of running your applications, grow your application's compute capacity and throughput for the same budget, and enable new types of cloud computing applications. Spot Instances are recommended for:
 - Applications that have flexible start and end times
 - Applications that are only feasible at very low compute prices
 - Users with urgent computing needs for large amounts of additional capacity
- **Reserved Instances** — [Reserved Instances](#) provide you with a significant discount (up to 72%) compared to On-Demand Instance pricing. You have the flexibility to change families, operating system types, and tenancies while benefiting from Reserved Instance pricing when you use Convertible Reserved Instances.

Elastic Load Balancing

[Elastic Load Balancing](#) (ELB) automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances, containers, and IP addresses. It can handle the varying load of your application traffic in a single Availability Zone or across multiple Availability Zones. Elastic Load Balancing offers four types of load balancers that all feature the high availability, automatic scaling, and robust security necessary to make your applications fault tolerant.

- [Application Load Balancer](#) is best suited for load balancing of HTTP and HTTPS traffic and provides advanced request routing targeted at the delivery of modern application architectures, including microservices and containers. Operating at the individual request level (Layer seven), Application Load Balancer routes traffic to targets within Amazon Virtual Private Cloud (Amazon VPC) based on the content of the request.
- [Network Load Balancer](#) is best suited for load balancing of TCP traffic where extreme performance is required. Operating at the connection level (Layer four), Network Load Balancer routes traffic to targets within Amazon Virtual Private Cloud (Amazon VPC) and is capable of handling millions of requests per second while maintaining ultra-low latencies. Network Load Balancer is also optimized to handle sudden and volatile traffic patterns.
- [Gateway Load Balancer](#) makes it easy to deploy, scale, and run third-party virtual networking appliances. Providing load balancing and auto scaling for fleets of third-party appliances, Gateway Load Balancer is transparent to the source and destination of traffic. This capability makes it well suited for working with third-party appliances for security, network analytics, and other use cases.
- [Classic Load Balancer](#) provides basic load balancing across multiple Amazon EC2 instances and operates at both the request level and connection level. Classic Load Balancer is intended for applications that were built within the EC2-Classic network. EC2-Classic was retired on August 15, 2022.

AWS Elastic Beanstalk

[AWS Elastic Beanstalk](#) is an easy-to-use service for deploying and scaling web applications and services developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, Nginx, Passenger, and Internet Information Services (IIS).

You can simply upload your code, and AWS Elastic Beanstalk automatically handles the deployment, from capacity provisioning, load balancing, and auto scaling to application health monitoring. At the same time, you retain full control over the AWS resources powering your application and can access the underlying resources at any time.

AWS Lambda

[AWS Lambda](#) lets you run code without provisioning or managing servers. You pay only for the compute time you consume—there is no charge when your code is not running. With Lambda, you can run code for virtually any type of application or backend service—all with zero administration. Just upload your code, and Lambda takes care of everything required to run and scale your code with high availability. You can set up your code to automatically run from other AWS services, or you can call it directly from any web or mobile app.

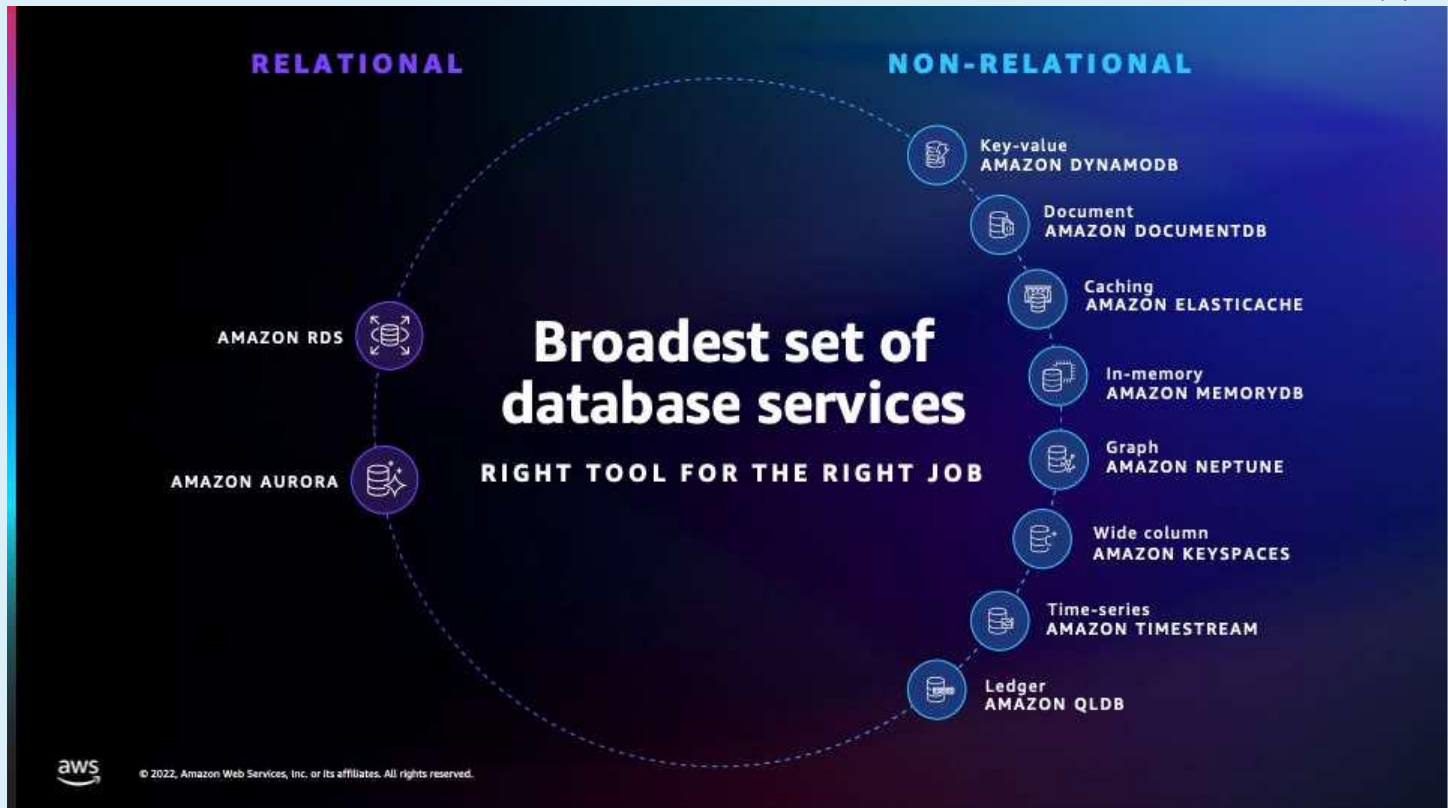
AWS Serverless Application Repository

The [AWS Serverless Application Repository](#) enables you to quickly deploy code samples, components, and complete applications for common use cases such as web and mobile backends, event and data processing, logging, monitoring, Internet of Things (IoT), and more. Each application is packaged with an [AWS Serverless Application Model](#) (AWS SAM) template that defines the AWS resources used. Publicly shared applications also include a link to the application's source code. There is no additional charge to use the AWS Serverless Application Repository - you only pay for the AWS resources used in the applications you deploy.

You can also use the AWS Serverless Application Repository to publish your own applications and share them within your team, across your organization, or with the community at large.

Databases





Amazon Aurora

[Amazon Aurora](#) is a MySQL and PostgreSQL compatible relational database engine that combines the speed and availability of high-end commercial databases with the simplicity and cost-effectiveness of open source databases.

Amazon Aurora is up to five times faster than standard MySQL databases and three times faster than standard PostgreSQL databases. It provides the security, availability, and reliability of commercial databases at 1/10th the cost. Amazon Aurora is fully managed by Amazon Relational Database Service (Amazon RDS), which automates time-consuming administration tasks such as hardware provisioning, database setup, patching, and backups.

Amazon Aurora features a distributed, fault-tolerant, self-healing storage system that auto-scales up to 128TB per database instance. It delivers high performance and availability with up to 15 low-latency read replicas, point-in-time recovery, continuous backup to Amazon S3, and replication across three Availability Zones (AZs).

Amazon Aurora I/O-Optimized is a cluster configuration that offers improved price performance and predictable pricing for customers with I/O-intensive applications, such as e-commerce applications, payment processing systems, and financial applications. Aurora-Optimized offers improved performance, increasing throughput and reducing latency to support your most demanding workloads, with up to 40 percent cost savings when your I/O spend exceeds 25 percent of your current Aurora database spend.

Amazon DynamoDB

[Amazon DynamoDB](#) is a key-value and document database that delivers single-digit millisecond performance at any scale. It's a fully managed, multi-Region database with built-in security, backup and restore, and in-memory caching for internet-scale applications. DynamoDB can handle more than 10 trillion requests per day and support peaks of more than 20 million requests per second.

Many of the world's fastest growing businesses such as Lyft, Airbnb, and Redfin, as well as enterprises such as Samsung, Toyota, and Capital One, depend on the scale and performance of DynamoDB to support their mission-critical workloads.

Hundreds of thousands of AWS customers have chosen DynamoDB as their key-value and document database for mobile, web, gaming, ad tech, Internet of Things (IoT), and other applications that need low-latency data access at any scale. Create a new table for your application and let DynamoDB handle the rest.

AWS Command Line Interface (AWS CLI)

The [AWS Command Line Interface](#) (AWS CLI) is a unified tool to manage your AWS services. With just one tool to download and configure, you can control multiple AWS services from the command line and automate them through scripts.

[AWS CloudShell](#), which can be found next to the search bar in the AWS Management Console, provides a browser-based shell that is pre-authenticated with your console credentials. Using CloudShell, you can quickly run AWS commands and scripts without leaving your web browser.

Amazon Redshift

[Amazon Redshift](#) is the most widely used cloud data warehouse. It makes it fast, simple and cost-effective to analyze all your data using standard SQL and your existing Business Intelligence (BI) tools. It allows you to run complex analytic queries against terabytes to petabytes of structured and semi-structured data, using sophisticated query optimization, columnar storage on high-performance storage, and massively parallel query completion. Most results come back in seconds.

You can start small for just \$0.25 per hour with no commitments and scale out to petabytes of data for \$1,000 per terabyte per year, less than a tenth the cost of traditional on-premises solutions.

AWS Data Pipeline

[AWS Data Pipeline](#) is a web service that helps you reliably process and move data between different AWS compute and storage services, as well as on-premises data sources, at specified

intervals. With AWS Data Pipeline, you can regularly access your data where it's stored, transform and process it at scale, and efficiently transfer the results to AWS services such as [Amazon S3](#), [Amazon RDS](#), [Amazon DynamoDB](#), and [Amazon EMR](#).

AWS Data Pipeline helps you easily create complex data processing workloads that are fault tolerant, repeatable, and highly available. You don't have to worry about ensuring resource availability, managing inter-task dependencies, retrying transient failures or timeouts in individual tasks, or creating a failure notification system. AWS Data Pipeline also allows you to move and process data that was previously locked up in on-premises data silos.

AWS Glue

[AWS Glue](#) is a fully managed extract, transform, and load (ETL) service that makes it easy for customers to prepare and load their data for analytics. You can create and run an ETL job with a few clicks in the AWS Management Console. You simply point AWS Glue to your data stored in AWS, and AWS Glue discovers your data and stores the associated metadata (such as table definition and schema) in the AWS Glue Data Catalog. Once cataloged, your data is immediately searchable, queryable, and available for ETL.

[AWS Glue Data Integration Engines](#) provide access to data using Apache Spark, PySpark, and Python. With the addition of AWS Glue for Ray, you can further scale your workloads using [Ray](#), an open-source unified compute framework.

[AWS Glue Data Quality](#) can measure and monitor the data quality of Amazon S3 based data lakes, data warehouses, and other data repositories. It automatically computes statistics, recommends quality rules, and can monitor and alert you when it detects missing, stale, or bad data. You can access it in the AWS Glue Data Catalog and in AWS Glue Data Catalog ETL jobs.

AWS Step Functions

[AWS Step Functions](#) is a fully managed service that makes it easy to coordinate the components of distributed applications and microservices using visual workflows. Building applications from individual components that each perform a discrete function lets you scale easily and change applications quickly. Step Functions is a reliable way to coordinate components and step through the functions of your application. Step Functions provides a graphical console to arrange and visualize the components of your application as a series of steps. This makes it simple to build and run multi-step applications. Step Functions automatically initiates and tracks each step, and retries when there are errors, so your application runs in order and as expected. Step Functions logs the state of each step, so when things do go wrong, you can diagnose and debug problems quickly. You can change and add steps without even writing code, so you can easily evolve your application and innovate faster.

Amazon Simple Notification Service

[Amazon Simple Notification Service](#) (Amazon SNS) is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and serverless applications. Amazon SNS provides topics for high-throughput, push-based, many-to-many messaging. Using Amazon SNS topics, your publisher systems can fan out messages to a large number of subscriber endpoints for parallel processing, including Amazon SQS queues, AWS Lambda functions, and HTTP/S webhooks. Additionally, SNS can be used to fan out notifications to end users using mobile push, SMS, and email.

AWS Cost Explorer

[AWS Cost Explorer](#) has an easy-to-use interface that lets you visualize, understand, and manage your AWS costs and usage over time. Get started quickly by creating custom reports (including charts and tabular data) that analyze cost and usage data, both at a high level (such as total costs and usage across all accounts) and for highly-specific requests (such as m2.2xlarge costs within account Y that are tagged “project: secretProject”).

AWS Cost and Usage Report

The [AWS Cost and Usage Report](#) is a single location for accessing comprehensive information about your AWS costs and usage.

The AWS Cost and Usage Report lists AWS usage for each service category used by an account and its IAM users in hourly or daily line items, as well as any tags that you have activated for cost allocation purposes. You can also customize the AWS Cost and Usage Report to aggregate your usage data to the daily or monthly level.

Developer tools



AWS CodeBuild

[AWS CodeBuild](#) is a fully managed build service that compiles source code, runs tests, and produces software packages that are ready to deploy. With CodeBuild, you don't need to provision, manage,

and scale your own build servers. CodeBuild scales continuously and processes multiple builds concurrently, so your builds are not left waiting in a queue. You can get started quickly by using

prepackaged build environments, or you can create custom build environments that use your own build tools.

AWS CodeCommit

[AWS CodeCommit](#) is a fully managed source control service that makes it easy for companies to host secure and highly scalable private Git repositories. AWS CodeCommit eliminates the need to operate your own source control system or worry about scaling its infrastructure. You can use AWS CodeCommit to securely store anything from source code to binaries, and it works seamlessly with your existing Git tools.

AWS CodeDeploy

[AWS CodeDeploy](#) is a service that automates code deployments to any instance, including EC2 instances and instances running on premises. CodeDeploy makes it easier for you to rapidly release new features, helps you avoid downtime during application deployment, and handles the complexity of updating your applications. You can use CodeDeploy to automate software deployments, eliminating the need for error-prone manual operations. The service scales with your infrastructure so you can easily deploy to one instance or thousands.

AWS CodePipeline

[AWS CodePipeline](#) is a fully managed continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates. CodePipeline automates the build, test, and deploy phases of your release process every time there is a code change, based on the release model you define. This enables you to rapidly and reliably deliver features and updates. You can easily integrate CodePipeline with third-party services such as

GitHub or with your own custom plugin. With AWS CodePipeline, you only pay for what you use. There are no upfront fees or long-term commitments.

AWS CloudFormation

[AWS CloudFormation](#) gives developers and systems administrators an easy way to create and manage a collection of related AWS resources, provisioning and updating them in an orderly and predictable fashion.

You can use the AWS CloudFormation [sample templates](#) or create your own templates to describe your AWS resources, and any associated dependencies or runtime parameters, required to run your application. You don't need to figure out the order for provisioning AWS services or the subtleties of making those dependencies work. CloudFormation takes care of this for you. After the AWS resources are deployed, you can modify and update them in a controlled and predictable way, in effect applying version control to your AWS infrastructure the same way you do with your software. You can also visualize your templates as diagrams and edit them using a drag-and-drop interface with [AWS Infrastructure Composer](#).

AWS CloudTrail

[AWS CloudTrail](#) is a web service that records AWS API calls for your account and delivers log files to you. The recorded information includes the identity of the API caller, the time of the API call, source IP

address of the API caller, the request parameters, and the response elements returned by the AWS service.

With CloudTrail, you can get a history of AWS API calls for your account, including API calls made using the AWS Management Console, AWS SDKs, command line tools, and higher-level AWS services (such as [AWS CloudFormation](#)). The AWS API call history produced by CloudTrail enables security analysis, resource change tracking, and compliance auditing.

Amazon CloudWatch

[Amazon CloudWatch](#) is a monitoring and management service built for developers, system operators, site reliability engineers (SRE), and IT managers. CloudWatch provides you with data and actionable insights to monitor your applications, understand and respond to system-wide performance changes, optimize resource utilization, and get a unified view of operational health. CloudWatch collects monitoring and operational data in the form of logs, metrics, and events, providing you with a unified view of AWS resources, applications and services that run on AWS, and on-premises servers. You can use CloudWatch to set high resolution alarms, visualize logs and metrics side by side, take automated actions, troubleshoot issues, and discover insights to optimize your applications, and ensure they are running smoothly.

Amazon API Gateway

[Amazon API Gateway](#) is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. With a few clicks in the AWS Management Console, you can create an API that acts as a “front door” for applications to access data, business logic, or functionality from your back-end services, such as workloads running on Amazon EC2, code running on AWS Lambda, or any web application. Amazon API Gateway handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls, including traffic management, authorization and access control, monitoring, and API version management.

Amazon CloudFront

[Amazon CloudFront](#) is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency, high transfer speeds, all within a developer-friendly environment. CloudFront is integrated with AWS – both physical locations that are directly connected to the AWS global infrastructure, as well as other AWS services. CloudFront works seamlessly with services including AWS Shield for DDoS

mitigation, Amazon S3, Elastic Load Balancing or Amazon EC2 as origins for your applications, and Lambda@Edge to run custom code closer to customers’ users and to customize the user experience.

You can get started with the content delivery network in minutes, using the same AWS tools that you’re already familiar with: APIs, AWS Management Console, AWS CloudFormation, CLIs, and SDKs. Amazon CDN offers a simple, pay-as-you-go pricing model with no upfront fees or required long-term contracts, and support for the CDN is included in your existing AWS Support subscription.

Amazon Route 53

[Amazon Route 53](#) is a highly available and scalable cloud Domain Name System (DNS) web service. It is designed to give developers and businesses an extremely reliable and cost-effective way to route users to internet applications by translating human-readable names, such as `www.example.com`, into the numeric IP addresses, such as `192.0.2.1`, that computers use to connect to each other. Amazon Route 53 is fully compliant with IPv6 as well.

Amazon Route 53 effectively connects user requests to infrastructure running in AWS—such as EC2 instances, elastic load balancers, or Amazon S3 buckets—and can also be used to route users to infrastructure outside of AWS. You can use Amazon Route 53 to configure DNS health checks to route traffic to healthy endpoints or to independently monitor the health of your application and its endpoint

